

AI Application

2주차: Python 데이터 처리 기초

■ 지난 주 복습 (1) — AI / ML / DL의 관계

- 인공지능(AI, Artificial Intelligence): 인간의 지능적 행동을 모방하는 기술 전반
 - 규칙 기반 시스템부터 학습 기반 시스템까지 포괄
- 머신러닝(ML, Machine Learning): 데이터로부터 패턴을 학습하는 AI의 하위 분야
 - 명시적 프로그래밍 없이 경험(데이터)으로 성능 개선
- 딥러닝(DL, Deep Learning): 다층 신경망(Neural Network)을 사용하는 ML의 하위 분야
 - 이미지, 음성, 자연어 등 비정형 데이터에 강점
- 포함 관계: $AI \supset ML \supset DL$

※ 이 관계는 시험에 반드시 나옵니다. 그림으로 그려서 기억하세요.

지난 주 복습 (2) — 모빌리티 분야의 AI 사례

- 자율주행(Autonomous Driving): 카메라/LiDAR 인식 → 판단 → 제어
- ADAS(첨단 운전자 보조 시스템): 차선 유지, 긴급 제동, 스마트 크루즈
- 교통 흐름 예측(Traffic Flow Prediction): 시간대별 교통량 예측·신호 최적화
- 차량 예지 정비(Predictive Maintenance): 센서 로그로 고장 사전 감지
- MaaS(Mobility as a Service): 수요 예측 기반 배차·경로 최적화
- 공통점: 모든 사례의 출발점은 '데이터 처리'

※ 오늘 배우는 NumPy/Pandas가 바로 이 데이터 처리의 표준 도구입니다.

오늘의 학습 목표

- NumPy 배열(ndarray)을 생성하고 인덱싱/슬라이싱으로 원하는 데이터를 추출할 수 있다
- 브로드캐스팅(Broadcasting)과 벡터화 연산(Vectorization)의 원리를 설명할 수 있다
- Pandas DataFrame으로 데이터를 읽고, 필터링·groupby 집계를 수행할 수 있다
- 결측치(Missing Value)를 탐지하고 적절한 방법으로 처리할 수 있다
- matplotlib으로 선/막대/산점도/히스토그램을 그릴 수 있다
- 정규화(Normalization)와 이상치(Outlier)의 개념을 이해한다
- 차량 주행 로그 데이터를 직접 집계·시각화해 본다 (시연 + 과제)

오늘의 목차

- **PART 1. NumPy — 배열 연산의 기초**
 - 배열 생성, 인덱싱/슬라이싱, 브로드캐스팅, 벡터화, 통계 함수
- **PART 2. Pandas — 표 형태 데이터 처리**
 - Series/DataFrame, 읽기, 필터링, groupby, 결측치 처리
- **PART 3. matplotlib — 데이터 시각화**
 - 선/막대/산점도/히스토그램, 그래프 꾸미기
- **PART 4. 데이터 전처리 개념 — 정규화와 이상치**

※ 퀴즈 → 시연 → 과제 1 안내 순서로 마무리합니다.

PART 1

NumPy — 배열 연산의 기초

Numerical Python: AI 데이터 처리의 출발점

NumPy란 무엇인가?

- NumPy(Numerical Python): 다차원 배열(ndarray)을 다루는 파이썬 핵심 라이브러리
- 왜 리스트(list) 대신 NumPy인가?
 - 속도: C로 구현되어 순수 파이썬 반복문보다 수십~수백 배 빠름
 - 메모리: 같은 타입(dtype)의 원소를 연속된 메모리에 저장
 - 문법: 반복문 없이 배열 전체에 한 번에 연산 적용 (벡터화)
- Pandas, PyTorch, OpenCV 등 대부분의 AI 라이브러리가 NumPy 기반
- 모빌리티 예시: 1초에 100번 기록되는 차량 속도 센서 → 하루 864만 개 값도 즉시 연산

배열 생성 (Array Creation)

```
import numpy as np

a = np.array([10, 20, 30, 40])      # 리스트에서 생성
z = np.zeros((2, 3))                # 0으로 채운 2x3 배열
o = np.ones(5)                      # 1로 채운 배열
r = np.arange(0, 10, 2)              # [0 2 4 6 8]
l = np.linspace(0, 1, 5)             # [0. 0.25 0.5 0.75 1.]
rand = np.random.rand(3)             # 0~1 균등분포 난수

print(a.shape)      # (4,)   배열의 모양
print(a.dtype)      # int64  원소의 자료형
print(a.ndim)       # 1      차원 수
```

※ shape, dtype, ndim은 배열을 받으면 가장 먼저 확인하는 3가지 속성입니다.

인덱싱과 슬라이싱 (Indexing / Slicing)

```
import numpy as np
speed = np.array([62, 58, 71, 45, 90, 83, 67]) # 차량 속도 (km/h)

speed[0]      # 62   첫 번째 원소
speed[-1]     # 67   마지막 원소
speed[1:4]    # [58 71 45] 1~3번 인덱스
speed[::2]    # [62 71 90 67] 2칸 간격

m = np.array([[1, 2, 3],
              [4, 5, 6]])
m[0, 2]       # 3     0행 2열
m[:, 1]       # [2 5]  모든 행의 1열
m[1, :]      # [4 5 6] 1행 전체
```

※ 2차원 배열은 [행, 열] 순서. 콜론(:)은 '전부'라는 뜻입니다.

불리언 인덱싱 (Boolean Indexing) — 조건으로 뽑기

```
import numpy as np
speed = np.array([62, 58, 71, 45, 90, 83, 67])

mask = speed > 70          # [False False True False True True False]
speed[mask]                # [71 90 83] → 과속(70 초과) 구간만 추출

# 한 줄로도 가능
speed[speed > 70]          # [71 90 83]

# 조건 결합: & (AND), | (OR) - 괄호 필수!
speed[(speed >= 60) & (speed <= 80)] # [62 71 67]

# 조건을 만족하는 개수
np.sum(speed > 70)         # 3 (True=1로 계산)
```

※ 모빌리티 예시: 과속 구간 검출, 급감속 이벤트 추출 등에 그대로 사용됩니다.

브로드캐스팅 (Broadcasting)

- 모양(shape)이 다른 배열끼리 연산할 때, 작은 배열을 자동으로 '확장'하는 규칙
- 가장 흔한 형태: 배열 ↔ 스칼라(숫자 하나)
 - `speed_kmh * 1000 / 3600` → 모든 원소를 m/s로 변환
- 배열 ↔ 배열: 뒤쪽 차원부터 비교하여 크기가 같거나 한쪽이 1이면 확장 가능
 - (3, 4) 배열 + (4,) 배열 → (4,)가 각 행에 복사되듯 적용
 - (3, 4) 배열 + (3,) 배열 → 오류! 뒤 차원 4 ≠ 3
- 장점: 복사본을 실제로 만들지 않아 메모리 효율적

※ 브로드캐스팅 오류(operands could not be broadcast)는 shape부터 확인하세요.

브로드캐스팅 예시 — 단위 변환과 보정

```
import numpy as np

speed_kmh = np.array([60.0, 80.0, 100.0])
speed_ms = speed_kmh * 1000 / 3600      # 스칼라 브로드캐스팅
# [16.67 22.22 27.78] (m/s)

# 3대 차량 x 4개 센서 오프셋 보정
raw = np.array([[1.2, 3.4, 2.2, 0.9],
                [1.1, 3.6, 2.0, 1.0],
                [1.3, 3.3, 2.4, 0.8]])
offset = np.array([0.1, 0.2, 0.0, 0.1]) # 센서별 보정값 (4,)
calibrated = raw - offset                # (3,4) - (4,) → OK
print(calibrated.shape)                  # (3, 4)
```

※ 센서 캘리브레이션(calibration)은 실제 차량 데이터 전처리의 첫 단계입니다.

벡터화 연산 (Vectorization) — 반복문을 버려라

```
import numpy as np
n = 1_000_000
speed = np.random.uniform(0, 120, n)  # 100만 개 속도 데이터

# (느림) 순수 파이썬 반복문
result = []
for s in speed:
    result.append(s * 1000 / 3600)

# (빠름) 벡터화 - 반복문 없이 한 줄
result = speed * 1000 / 3600

# 조건 연산도 벡터화: np.where(조건, 참일 때, 거짓일 때)
label = np.where(speed > 100, "과속", "정상")
```

※ 같은 결과, 속도는 수십~수백 배 차이. AI 코드에서 for문이 보이면 일단 의심하세요.

주요 통계 함수 (Statistical Functions)

함수	의미	차량 속도 로그 예시
np.mean(a)	평균	평균 주행 속도
np.median(a)	중앙값	이상치에 강한 대표 속도
np.std(a)	표준편차	속도 변동성(주행 안정성)
np.min / np.max	최솟값 / 최댓값	최저·최고 속도
np.percentile(a, 95)	백분위수	상위 5% 속도 기준선
np.sum / np.cumsum	합 / 누적합	총 주행거리 근사
a.argmax()	최댓값의 위치	최고 속도가 나온 시각

※ axis=0(열 방향), axis=1(행 방향) 인자로 2차원 배열의 방향별 통계도 계산합니다.

통계 함수 실전 — 주행 데이터 요약

```
import numpy as np
np.random.seed(0)
speed = np.random.normal(65, 12, 3600)  # 1시간(1초 간격) 속도 로그
speed = np.clip(speed, 0, None)          # 음수 속도 제거

print("평균 속도 :", np.mean(speed).round(1), "km/h")
print("중앙값   :", np.median(speed).round(1), "km/h")
print("표준편차  :", np.std(speed).round(1))
print("최고 속도 :", np.max(speed).round(1), "km/h")
print("95백분위  :", np.percentile(speed, 95).round(1))
print("과속 비율 :", (speed > 80).mean().round(3) * 100, "%")

# 2차원: 차량 5대 x 60분 → 차량별 평균
fleet = np.random.normal(60, 10, (5, 60))
print(fleet.mean(axis=1))          # 차량별(행별) 평균 5개
```

※ (조건).mean()은 True 비율 = '조건 만족 비율'을 구하는 관용 표현입니다.

NumPy 핵심 한 줄 요약

"반복문 대신 배열 전체에 연산한다" — shape을 이해하면 NumPy의 90%를 이해한 것입니다.

PART 2

Pandas — 표 형태 데이터 처리

엑셀보다 강력한 파이썬 데이터 분석 도구

Pandas란 무엇인가?

- Pandas: 표(테이블) 형태 데이터를 다루는 파이썬 라이브러리 (Panel Data에서 유래)
- Series: 1차원 — 인덱스(index)가 붙은 배열 (엑셀의 한 열)
- DataFrame: 2차원 — 여러 Series의 모음 (엑셀 시트 전체)
- NumPy와의 차이
 - NumPy: 같은 타입의 숫자 배열, 위치(정수) 인덱스
 - Pandas: 열마다 다른 타입 가능(문자+숫자+시각), 이름(라벨) 인덱스
- 모빌리티 예시: [시각, 차량ID, 속도, 연비, 도로유형] 같은 주행 로그 테이블

Series와 DataFrame 만들기

```
import pandas as pd

# Series: 값 + 인덱스
s = pd.Series([62, 58, 71], index=["09:00", "09:01", "09:02"])
s["09:01"]          # 58 - 라벨로 접근

# DataFrame: 딕셔너리 → 표
df = pd.DataFrame({
    "vehicle_id": ["V01", "V02", "V01", "V03"],
    "speed":      [62.5, 58.1, 71.3, 45.0],
    "fuel_eff":   [12.3, 14.1, 10.8, 15.2],    # km/L
    "road":       ["고속도로", "시내", "고속도로", "시내"],
})
print(df.shape)      # (4, 4)
print(df.dtypes)     # 열별 자료형
```

※ 실무에서는 직접 만들기보다 CSV 파일을 읽는 경우가 대부분입니다.

데이터 읽기와 훑어보기

```
import pandas as pd

df = pd.read_csv("driving_log.csv")      # CSV 읽기
# df = pd.read_excel("log.xlsx")        # 엑셀도 가능

df.head()          # 앞 5행 미리보기
df.tail(3)         # 뒤 3행
df.info()          # 행 수, 열 타입, 결측치 개수 요약
df.describe()      # 숫자 열의 평균/표준편차/사분위수
df.shape           # (행 수, 열 수)
df.columns         # 열 이름 목록
df["road"].value_counts() # 범주형 열의 빈도
```

※ 새 데이터를 받으면 head → info → describe 3단 콤보가 국룰입니다.

열 선택과 행 필터링 (Filtering)

```
# 열 선택
df["speed"]          # 한 열 → Series
df[["speed", "fuel_eff"]] # 여러 열 → DataFrame (대괄호 2접!)

# 행 필터링: 불리언 인덱싱 (NumPy와 동일한 원리)
df[df["speed"] > 60]          # 60km/h 초과 행
df[df["road"] == "고속도로"]  # 고속도로 주행만
df[(df["speed"] > 60) & (df["fuel_eff"] > 12)] # AND 조건

# loc: 라벨 기반, iloc: 위치 기반
df.loc[0, "speed"]          # 0번 라벨 행의 speed 값
df.iloc[0:3, 1:3]          # 위치로 슬라이싱

# 새 열 추가 (벡터화 연산)
df["speed_ms"] = df["speed"] * 1000 / 3600
```

※ 조건 결합 시 각 조건을 반드시 괄호로 감싸야 합니다 — 가장 흔한 초보 오류.

groupby — 그룹별 집계와 집계 함수

- groupby: '나누고(Split) → 계산하고(Apply) → 합친다(Combine)' 패턴
 - 예: 도로 유형별로 나눠 → 각 그룹의 평균 속도 계산 → 하나의 표로
- 기본 형태: `df.groupby("기준열")["대상열"].집계함수()`
- 집계 함수: `mean, sum, count, min, max, std, median ...`
- 여러 집계를 한 번에: `.agg(["mean", "max", "count"])`
- 두 개 이상의 기준: `df.groupby(["vehicle_id", "road"])`
- 모빌리티 예시: 시간대별 교통량 집계, 차량별 평균 연비, 도로유형별 과속 비율

※ SQL의 GROUP BY, 엑셀의 피벗테이블과 같은 개념입니다.

groupby 실전 — 시간대별 교통량 집계

```
import pandas as pd

# 교차로 통과 차량 로그 (시각, 차량ID)
df["hour"] = pd.to_datetime(df["timestamp"]).dt.hour

# 시간대별 교통량 (통과 차량 수)
traffic = df.groupby("hour")["vehicle_id"].count()
# hour
# 7      412
# 8      977  ← 출근 첨두시간(peak hour)
# 9      603 ...

# 도로유형별 평균 속도와 연비를 한 번에
summary = df.groupby("road").agg(
    mean_speed=("speed", "mean"),
    mean_fuel=("fuel_eff", "mean"),
    n=("speed", "count"),
)
```

※ named aggregation: 결과 열 이름을 직접 지정하는 권장 문법입니다.

결측치 (Missing Value) — 왜 생기고 왜 문제인가

- 결측치(NaN, Not a Number): 값이 기록되지 않은 칸
- 발생 원인 (모빌리티 현장)
 - 센서 순간 단절, GPS 음영 지역(터널), 통신 패킷 손실
 - 서로 다른 주기의 센서 병합(10Hz 속도 + 1Hz GPS)
- 문제점: 평균 왜곡, 대부분의 ML 모델은 NaN 입력 시 오류
- 처리 전략 3가지
 - 삭제(dropna): 결측 행이 적을 때
 - 대체(fillna): 평균/중앙값/직전 값(ffill)으로 채움
 - 그대로 두기: 결측 자체가 정보인 경우 (예: 터널 구간 표시)

결측치 탐지와 처리

```
import pandas as pd

df.isna().sum()          # 열별 결측치 개수
df.isna().mean() * 100   # 열별 결측 비율(%)

# 전략 1: 삭제
df_drop = df.dropna()    # 결측 있는 행 전부 삭제
df_drop = df.dropna(subset=["speed"]) # speed가 NaN인 행만 삭제

# 전략 2: 대체
df["speed"] = df["speed"].fillna(df["speed"].mean()) # 평균 대체
df["gps_lat"] = df["gps_lat"].ffill() # 직전 값으로 채움
# → 터널에서 끊긴 GPS는 ffill이 자연스러움 (시계열 데이터)

df.isna().sum().sum()    # 전체 결측치 0인지 최종 확인
```

※ 시계열 센서 데이터는 평균 대체보다 ffill(forward fill)이 적합한 경우가 많습니다.

NumPy vs Pandas 비교 정리

구분	NumPy	Pandas
기본 자료구조	ndarray (동일 타입)	Series / DataFrame (열별 타입)
인덱스	정수 위치	라벨(이름) + 위치
강점	수치 연산 속도	표 데이터 조작·집계
결측치	np.nan (제한적)	isna/fillna/dropna 완비
파일 입출력	npy, txt	CSV, Excel, SQL, JSON
대표 용도	센서 신호 연산, 이미지 배열	주행 로그 테이블 분석

※ 둘은 경쟁 관계가 아니라 df.values로 오가는 상호보완 관계입니다.

Pandas 핵심 한 줄 요약

"필터링과 groupby만 자유로워지면 실무 데이터 분석의 절반은 끝" — 오늘 과제에서 직접 확인합니다.

PART 3

matplotlib — 데이터 시각화

숫자를 그림으로: 보이지 않던 패턴이 보인다

왜 시각화(Visualization)인가?

- 통계 수치만으로는 놓치는 패턴을 눈으로 발견 (분포, 추세, 이상치)
- Anscombe의 4중주: 평균·분산·상관계수가 같아도 그림은 완전히 다를 수 있다
- matplotlib: 파이썬 시각화의 표준 라이브러리 (pyplot 인터페이스 사용)
- 기본 4대 그래프와 용도
 - 선 그래프(line): 시간에 따른 변화 — 속도 프로파일
 - 막대 그래프(bar): 범주 간 비교 — 시간대별 교통량
 - 산점도(scatter): 두 변수의 관계 — 속도 vs 연비
 - 히스토그램(histogram): 분포 확인 — 속도 분포와 이상치

선 그래프 — 시간에 따른 속도 변화

```
import matplotlib.pyplot as plt
import numpy as np

t = np.arange(0, 600) # 10분 (초 단위)
speed = 60 + 15 * np.sin(t / 60) + np.random.randn(600) * 3

plt.figure(figsize=(10, 4))
plt.plot(t, speed, color="steelblue", linewidth=1)
plt.axhline(80, color="red", linestyle="--", label="제한속도 80")
plt.xlabel("Time (s)")
plt.ylabel("Speed (km/h)")
plt.title("Vehicle Speed Profile")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

※ axhline으로 기준선(제한속도)을 함께 그리면 해석이 쉬워집니다.

막대 그래프 — 시간대별 교통량

```
import matplotlib.pyplot as plt

hours = [6, 7, 8, 9, 10, 17, 18, 19]
volume = [220, 640, 980, 610, 380, 870, 1020, 700]

plt.figure(figsize=(9, 4))
plt.bar(hours, volume, color="teal")
plt.xlabel("Hour of Day")
plt.ylabel("Traffic Volume (vehicles/h)")
plt.title("Hourly Traffic Volume at Intersection A")
plt.show()

# Pandas와 연동: groupby 결과를 바로 그리기
# df.groupby("hour")["vehicle_id"].count().plot(kind="bar")
```

※ 출근(8시)·퇴근(18시) 첨두를 한눈에 — 신호 최적화의 근거 데이터가 됩니다.

산점도 — 속도와 연비의 관계

```
import matplotlib.pyplot as plt
import numpy as np

speed = np.random.uniform(20, 120, 300)
fuel = 18 - 0.002 * (speed - 65) ** 2 + np.random.randn(300)

plt.figure(figsize=(7, 5))
plt.scatter(speed, fuel, s=12, alpha=0.5, color="darkorange")
plt.xlabel("Speed (km/h)")
plt.ylabel("Fuel Efficiency (km/L)")
plt.title("Speed vs Fuel Efficiency")
plt.grid(alpha=0.3)
plt.show()
# → 약 65km/h 부근에서 연비 최적 (경제속도)라는 패턴 발견
```

※ 산점도는 '두 변수의 관계'를 보는 도구 — 3주차 회귀(Regression)로 이어집니다.

히스토그램 — 속도 분포와 이상치

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(1)
speed = np.random.normal(65, 12, 2000)
speed[:200] = 180          # 센서 오류로 튕 값(이상치) 삽입

plt.figure(figsize=(8, 4))
plt.hist(speed, bins=40, color="slateblue", edgecolor="white")
plt.xlabel("Speed (km/h)")
plt.ylabel("Count")
plt.title("Speed Distribution (1h log)")
plt.show()
# → 오른쪽 끝 180 부근의 외판 막대 = 이상치가 한눈에 보임
```

※ bins(구간 수)를 바꿔 보면서 분포의 모양을 확인하는 습관을 들이세요.

여러 그래프 한 화면에 — subplot

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(t, speed_profile)
axes[0].set_title("Speed Profile")
axes[0].set_xlabel("Time (s)")

axes[1].hist(speed_profile, bins=30, color="teal")
axes[1].set_title("Speed Distribution")
axes[1].set_xlabel("Speed (km/h)")

plt.tight_layout()      # 겹침 자동 정리
plt.show()
```

※ 같은 데이터를 '시간 축'과 '분포'로 동시에 보는 것이 센서 로그 분석의 기본입니다.

그래프 선택 가이드

보고 싶은 것	그래프	함수	모빌리티 예시
시간에 따른 변화	선 그래프	plt.plot()	주행 중 속도/배터리 변화
범주 간 크기 비교	막대 그래프	plt.bar()	시간대별·요일별 교통량
두 변수의 관계	산점도	plt.scatter()	속도 vs 연비, 온도 vs 배터리
값의 분포	히스토그램	plt.hist()	속도 분포, 배차 대기시간 분포

※ "무엇을 보여주고 싶은가"가 먼저, 그래프 종류는 그 다음입니다.

좋은 그래프의 조건 (실습·과제 채점 기준이기도 함)

- 제목(title)과 축 라벨(xlabel/ylabel) + 단위는 필수
 - 'Speed'가 아니라 'Speed (km/h)' — 단위 없는 그래프는 감점
- 범례(legend): 선이 2개 이상이면 반드시 표시
- 기준선·강조: axhline, 색상으로 메시지를 명확히
- 과유불급: 3D, 화려한 색, 불필요한 격자는 오히려 방해
- 한글 폰트 문제: 그래프 라벨은 영문 사용 권장 (환경 독립적)

시각화 핵심 한 줄 요약

"describe()로 요약하고, 그림으로 확인한다" — 숫자와 그림은 항상 세트입니다.

PART 4

데이터 전처리 (Preprocessing)

정규화와 이상치: 모델에 넣기 전 마지막 관문

전처리가 왜 중요한가?

- "Garbage In, Garbage Out" — 나쁜 데이터로는 좋은 모델이 불가능
- 실무 AI 프로젝트 시간의 60~80%가 데이터 수집·전처리
- 전처리의 주요 단계
 - 결측치 처리 (PART 2에서 학습)
 - 이상치 탐지·처리 (Outlier Detection)
 - 스케일링: 정규화(Normalization) / 표준화(Standardization)
 - 범주형 변수 인코딩 (3주차에서 계속)
- 모빌리티 예시: 속도(0~120)와 GPS 위도(37.xx)를 그대로 섞으면 모델이 속도만 쳐다봄

정규화 (Normalization) vs 표준화 (Standardization)

- 왜 필요한가: 변수마다 단위·범위가 다르면 큰 숫자가 학습을 지배
- Min-Max 정규화: $(x - \min) / (\max - \min) \rightarrow 0 \sim 1$ 범위로 변환
 - 장점: 범위가 명확, 신경망 입력에 자주 사용
 - 단점: 이상치 하나가 min/max를 왜곡
- Z-score 표준화: $(x - \text{mean}) / \text{std} \rightarrow \text{평균 } 0, \text{표준편차 } 1$
 - 장점: 이상치 영향이 상대적으로 작음, 통계 기법과 궁합
- 주의: 반드시 '훈련 데이터 기준'으로 변환값을 계산 (3주차에서 재등장)

정규화·표준화 구현 — NumPy 한 줄씩

```
import numpy as np

speed = np.array([45.0, 62.0, 71.0, 88.0, 120.0]) # km/h
odo = np.array([12000, 45000, 3000, 78000, 30000]) # 누적주행 km

# Min-Max 정규화: 0~1
speed_mm = (speed - speed.min()) / (speed.max() - speed.min())
# [0.    0.227 0.347 0.573 1.    ]

# Z-score 표준화: 평균 0, 표준편차 1
odo_z = (odo - odo.mean()) / odo.std()

print(speed_mm.round(3))
print(odo_z.round(3))
# 이제 두 변수가 비슷한 스케일 → 공정하게 학습에 기여
```

※ 공식은 단 한 줄 — 브로드캐스팅 덕분에 반복문이 필요 없습니다.

이상치 (Outlier) — 정의와 원인

- 이상치: 다른 관측값들과 동떨어진 값
- 원인 구분이 중요
 - 오류 이상치: 센서 고장, 통신 오류 → 제거/보정 대상
 - 진짜 이상치: 실제 급가속, 사고 상황 → 오히려 중요한 신호!
- 모빌리티 예시
 - GPS 튜: 시내 주행 중 속도 300km/h 기록 → 명백한 오류
 - 급제동 -9.8m/s^2 : 오류가 아니라 사고 위험 이벤트일 수 있음
- 따라서 '무조건 삭제'가 아니라 원인을 확인 후 처리 방법 결정

이상치 탐지 — IQR 방법

```
import numpy as np

speed = np.array([58, 62, 65, 61, 63, 300, 59, 64, 60, 66])

q1 = np.percentile(speed, 25)      # 1사분위수
q3 = np.percentile(speed, 75)      # 3사분위수
iqr = q3 - q1                      # 사분위 범위(IQR)

lower = q1 - 1.5 * iqr             # 하한선
upper = q3 + 1.5 * iqr            # 상한선

outliers = speed[(speed < lower) | (speed > upper)]
print(outliers)                   # [300] ← GPS 튜 검출

clean = speed[(speed >= lower) & (speed <= upper)]
print(clean.mean())               # 이상치 제거 후 평균 62.0
```

※ 박스플롯(boxplot)의 수염이 바로 이 $1.5 \times \text{IQR}$ 기준입니다.

이상치 처리 전략 비교

제거 / 대체가 적절한 경우

- 물리적으로 불가능한 값 (속도 300km/h)
- 센서 스펙 범위를 벗어난 값
- 중복 기록·타임스탬프 역행
- 처리 방법
 - 행 삭제 (오류가 드물 때)
 - 경계값으로 자르기: `np.clip()`
 - 직전 값·중앙값으로 대체

보존해야 하는 경우

- 실제 발생한 극단 이벤트 (급제동, 사고)
- 탐지 자체가 목적인 경우 (이상 탐지 모델)
- 표본이 적어 삭제 시 정보 손실이 큰 경우
- 처리 방법
 - 별도 플래그 열 추가 후 보존
 - 이상치에 강한 통계 사용 (중앙값)
 - 도메인 전문가와 상의

전처리 파이프라인 정리 — 오늘 배운 것의 연결

- 1단계: 데이터 로드 — `pd.read_csv()`
- 2단계: 훑어보기 — `head` / `info` / `describe`
- 3단계: 결측치 처리 — `isna` → `dropna` / `fillna` / `ffill`
- 4단계: 이상치 처리 — 히스토그램·IQR로 탐지 → 원인 판단 후 처리
- 5단계: 스케일링 — Min-Max 또는 Z-score
- 6단계: 시각화로 최종 확인 — 처리 전/후 분포 비교
- 이 파이프라인이 3주차 머신러닝의 입력 데이터를 만듭니다

※ 순서가 중요합니다: 이상치를 처리하기 전에 정규화하면 min/max가 왜곡됩니다.

전처리 핵심 한 줄 요약

"모델보다 데이터가 먼저" — 전처리 품질이 AI 성능의 상한선을 결정합니다.

퀴즈 1 — NumPy

Q. `speed = np.array([62, 58, 71, 45, 90])`일 때, `speed[speed > 60]`의 결과는?

- ① `[62, 71, 90]`
- ② `[True, False, True, False, True]`
- ③ `[58, 45]`
- ④ 오류 발생 (리스트가 아니므로)

▶ 정답: ① — 불리언 인덱싱은 조건이 True인 원소만 추출합니다. ②는 mask 자체(`speed > 60`)의 결과.

퀴즈 2 — Pandas

Q. 도로유형(road)별 평균 속도(speed)를 구하는 올바른 코드는?

- ① `df.groupby("speed")["road"].mean()`
- ② `df.groupby("road")["speed"].mean()`
- ③ `df["road"].mean("speed")`
- ④ `df.mean().groupby("road")`

▶ 정답: ② — `groupby`(나누는 기준 열)[계산할 열].집계함수() 순서를 기억하세요.

퀴즈 3 — 전처리

Q. 센서 로그에서 속도 300km/h(시내 주행 중)가 발견되었다. 가장 적절한 첫 대응은?

- ① 무조건 행을 삭제한다
- ② 평균값으로 즉시 대체한다
- ③ 히스토그램·IQR로 확인하고 물리적으로 불가능한 오류인지 원인을 판단한다
- ④ 정규화를 먼저 수행한다

▶ 정답: ③ — 이상치는 '원인 판단'이 먼저. 정규화를 먼저 하면(④) 이상치가 min/max를 왜곡합니다.

시연 (Live Demo) — Week02_demo.ipynb

- **지금부터 노트북으로 전체 흐름을 실습합니다 (약 40분)**
- **시나리오: 합성 차량 주행 로그 24시간 분석**
 - 1) NumPy로 주행 로그 데이터 생성 (시각, 속도, 연비, 도로유형)
 - 2) NumPy 통계·불리언 인덱싱으로 과속 구간 분석
 - 3) DataFrame 변환 → 필터링 → 시간대별 groupby 집계
 - 4) 결측치 주입 후 탐지·처리 실습
 - 5) matplotlib 4종 그래프로 시각화
- **노트북 파일은 Google Drive 공유 폴더에서 다운로드 — 같이 실행하면서 따라오세요**

시연 환경 준비 (지금 확인하세요)

- **Google Colab 사용 시:** 브라우저에서 colab.research.google.com 접속 → 업로드
 - NumPy/Pandas/matplotlib 기본 설치되어 있음 — 준비 끝
- **로컬 환경(Anaconda) 사용 시**
 - `conda list numpy pandas matplotlib` 으로 설치 확인
 - 없다면: `pip install numpy pandas matplotlib`
- **실행이 막히는 학생은 옆 사람 화면을 같이 보며 진행 (조교 없음!)**
- **shift + Enter = 셀 실행.** 위에서부터 순서대로 실행하는 것이 원칙

오늘의 정리

- ✓ NumPy: shape 이해 + 불리언 인덱싱 + 브로드캐스팅 + 벡터화가 핵심
- ✓ Pandas: head/info/describe로 훑고, 필터링과 groupby로 집계한다
- ✓ 결측치는 탐지(isna) → 전략 선택(dropna/fillna/ffill) 순서로
- ✓ 그래프 4종(선/막대/산점도/히스토그램)의 용도를 구분해서 사용한다
- ✓ 정규화/표준화로 스케일을 맞추고, 이상치는 원인 판단이 먼저
- ✓ 이 모든 것이 차량 센서 로그·교통 데이터 분석의 기본기다

다음 주: 3주차 — 머신러닝 기초 I (지도학습, 회귀, train/test 분할)

과제 1 안내 — 데이터 처리 (빈칸 채우기 노트북)

- 파일: 과제1_데이터처리.ipynb (Google Drive 공유 폴더에서 다운로드)
- 형식: 빈칸 채우기(fill-in-the-blank) 7문제
 - 코드 셀의 __ 부분을 채우고 실행 → 바로 아래 자동채점 셀 실행
 - "문제 N 통과!"가 출력되면 정답 — 스스로 확인 가능
- 구성: 기본 5문제 (NumPy 2 + Pandas 2 + 전처리 1) + 심화 2문제(선택)
- 내용: 오늘 시연과 동일한 차량 주행 로그 데이터 사용
- 심화 문제는 가산점 — 기본 문제만 다 맞아도 만점

과제 1 — 제출 방법과 기한

- **제출 기한: 다음 주 수업 전날(화요일) 23:59까지**
- **제출 방법**
 - 모든 셀을 위에서부터 실행한 상태로 저장 (Runtime → Run all 권장)
 - 파일명: 과제1_학번_이름.ipynb 으로 변경
 - ① 본인 GitHub 저장소에 push (커밋 기록이 곧 제출 증빙)
 - ② e-Campus 과제 게시판에 같은 파일 제출
 - ③ 이메일로 알림 — 제목: [AI응용] 과제1_학번_이름, 본문에 저장소 링크
 - GitHub 사용이 어려우면 .ipynb를 이메일에 첨부해 제출해도 인정 (e-Campus 제출은 공통 필수)
- **채점: 자동채점 셀 통과 여부 (기본 5문제 x 20점, 심화는 가산점)**
- **주의: 자동채점 셀(assert)은 절대 수정 금지 — 수정 시 0점 처리**
- **질문: Mattermost 채널 활용 (에러 메시지 전체를 캡처해서 올릴 것)**

※ 다음 주 3주차: 머신러닝 기초 I — 오늘 만든 전처리 데이터가 그대로 입력이 됩니다.